Applied AI Associate (Public Health) — Take-Home Simulation Exercise

Role under assessment: Applied AI Associate (Public Health), Health Analytics and AI Unit (HAAU), embedded at the NHA. Also used to assess Digital Health Integration Specialist (Applied AI) candidates.

Format: Take-home, self-paced · **Window:** 24 hours from release · **Effort target:** 3–5 focused hours (do not spend the full 24) · **Submission:** a `.zip` export of your repo (and optionally a private repo link), plus a short `SUBMISSION.md` cover sheet (Section 8) · **AI use:** explicitly permitted and encouraged — see Section 7

---

0 · Read this first

You are not being asked to build a production system. You are being asked to show, in a compressed window, that you can take a public-health question end to end — provision the data, form a sharp hypothesis, test it honestly, and hand a decision-maker something they can act on — and to do it hands-on, not by describing what a team "would" do.

Everything here uses **entirely synthetic data** that loosely mimics AB PM-JAY claims, empanelled-hospital, and beneficiary-enrolment data. Nothing here touches real NHA data, and you must not use any. We care about four things, in this order:

- **Hands-on execution** — you can actually make the thing work and produce a result, not just sketch an approach.
- **Problem-solving under messy data** — when the data or tooling fights you, what you do about it.
- **Honest analytical judgement** — a well-formed hypothesis, a defensible finding, and a clear sense of what you would *not* claim from synthetic, dirty data.
- **On-ground critical thinking** — you understand the NHA context: what a senior official actually needs, and where the privacy and interpretation landmines are.

A polished half is worth more than a broken whole. If you run short of time, cut scope deliberately and say so — telling us what you would do next, and why, is itself a scored signal.

1 · The scenario

The HAAU is the NHA's in-house unit for advanced analytics and AI. A recurring reality of the unit is that programmatic data arrives as flat files that no one has yet turned into insight. Leadership hands you a synthetic extract and a blunt question:

> "We have claims and facility data sitting in flat files. Somewhere in here is a signal we could act on — but we can't see it, and we don't want a number we can't defend. Show

> us, this week, one thing worth knowing and the decision it points to. Small is fine. Fake-but-working beats real-but-broken."

Your job over the next few hours is to produce that single, clean end-to-end thread: **provision data → frame a hypothesis → test it → show the result and the decision it informs.** The steps are laid out in Section 3.

2 · A little domain context (no prior expertise required)

AB PM-JAY (Ayushman Bharat Pradhan Mantri Jan Arogya Yojana) is India's national publicly-funded health-assurance scheme. It covers eligible families for a defined annual amount toward hospitalisation at **empanelled** hospitals — public or private facilities that have been approved to treat PM-JAY beneficiaries and raise claims against the scheme. Eligible families are enrolled and issued an **Ayushman card**. Care is billed as standardised benefit packages (each with a package code and a set rate), and hospitals raise claims that are then adjudicated (approved / rejected / pending). Entitlement is **family-based**, so the family ID matters as much as the individual, and beneficiaries can seek care outside their home district or state.

A facility carries two independent flags that matter here: an **empanelment status** (is it enrolled in AB PM-JAY — approved to treat PM-JAY beneficiaries and raise claims?) and an **activity status** (is it currently operating?). The HAAU works with metadata-level and programmatic scheme data — for example, how enrolment compares with actual utilisation, or how actively empanelled hospitals participate — under the DPDP Act 2023 and the Strategy for Artificial Intelligence in Healthcare for India (SAHI).

The three synthetic files in Section 3 loosely mirror this world. This is the framing your analysis should speak to.

3 · The task, step by step

This is **one** end-to-end thread, not several tasks. Work through the six steps in order. Each deliverable carries an ID (D1, D2, …) — use these exact IDs as your file names or section headings. The full list is in Section 8.

Step 1 — The problem statement (fixed)

> "Across districts, where is the scheme's **reach** (how many eligible people hold an Ayushman card) most out of step with its **use** (how much hospitalisation those districts actually claim) — and can that gap be explained by **facility supply** (the number of empanelled, active hospitals available)? Which districts should the NHA look at first, and should the fix be more supply, or more demand-side outreach?"

This question is answered entirely at the **district level** — you do not need to expose any individual record to answer it.

Step 2 — Break the problem down (D1)

Before touching code, decompose the question into something testable. In a short written breakdown, set out: the sub-questions you'll answer; the metrics you'll construct (e.g. cards-per-eligible-population, claims-per-card, empanelled-active-hospitals per 100k); the confounders and data-quality traps you expect; and what a "so-what" answer would look like for a senior official. This breakdown is scored — it shows us how you think before the tools do.

Step 3 — Generate your own synthetic data (D2)

You provision your own dataset — this is deliberate: creating realistic data is part of the test, and it removes any privacy risk. Write a generator script that produces at least the three files below.

File	Min. size	Fields (at minimum)
claims.csv	⩾ 8,000 rows	claim_id, beneficiary_id, family_id, district_id, state_id, hospital_id, package_code, package_name, specialty, claim_amount, approved_amount, date_admitted, date_discharged, claim_status (approved/rejected/pending), beneficiary_age, beneficiary_gender
facilities.csv	⩾ 150 hospitals	hospital_id, hospital_name, district_id, state_id, facility_type (public/private), empanelment_status (empanelled/de-empanelled), activity_status (active/dormant), bed_count, lat, lon
beneficiaries.csv	⩾ 5,000 rows	beneficiary_id, family_id, district_id, state_id, age, gender — the register of enrolled beneficiaries (Ayushman card holders). *

Note: empanelment status (is the facility enrolled in AB PM-JAY — approved to treat PM-JAY beneficiaries and raise claims?) and activity status (is it currently operating?) are two different things — a facility can be empanelled yet dormant, or active yet recently de-empanelled. Keep them as separate fields.

* To compute a coverage ratio you also need a small reference lookup of **eligible population by district** (`district_id → eligible_population`). Keep this as a separate small file or a dictionary in your generator — it is a reference table, not a column in `beneficiaries.csv`.

Seed realistic "dirt" on purpose — cleaning messy programmatic data is core to the role. Plant, for example: duplicate claim rows (the same `claim_id` appearing more than once); claims raised against hospitals that are **dormant** or **de-empanelled**; impossible dates (discharge before admission) and impossible amounts (`approved_amount` greater than `claim_amount`, or negative amounts); implausible `beneficiary_age` values; missing `district_id`, `package_code`, or dates; and at least one district where recorded card-holders

exceed its eligible population (a coverage rate above 100%). **Document what you seeded in `SEEDED_ISSUES.md`** — we will check whether your own analysis catches your planted problems. Using Faker, an LLM, or a similar generator is fine, as long as the output matches the schemas and contains the seeded issues.

Step 4 — State your hypothesis (D3)

Commit to a falsifiable hypothesis *before* you analyse, and write it down. A strong one is specific and directional — for example: "Districts in the lowest quartile of empanelled-active-hospital supply account for a disproportionate share of the reach-vs-use gap, so the binding constraint there is supply, not demand." State what result would **confirm** it and what would **refute** it. You are free to disprove your own hypothesis — an honest null result, clearly argued, scores as well as a positive one.

Step 5 — Prove (or disprove) it through analysis (D4)

Load the three files into a queryable store (a local DuckDB or SQLite file, an in-memory pandas pipeline, or a small Postgres — your choice; nothing paid) and test the hypothesis. Along the way: clean the data and show that your cleaning catches the issues you seeded; construct your metrics; and quantify the relationship between the reach-vs-use gap and facility supply (a ranking, a correlation, a simple segmentation, or a light model — sophistication is less important than being *right* and *legible*).

Step 6 — Show the result, the insight, and the decision (D5, D6)

Turn the analysis into something a non-technical senior official can absorb in one screen (D5): a small dashboard, a static chart set, or a single HTML/PDF page — a ranked district shortlist plus one or two supporting visuals is plenty. Then write a **≤ 250-word note to leadership** (D6) that states, in plain language: the hypothesis you tested; what the number is; which districts to look at first and whether the lever is supply or demand; and — critically — **what you would not claim** from synthetic, dirty data. On-ground judgement is scored here as much as the code.

4 · What "good" looks like (self-check)

- The thread actually runs from your repo on our machine, from a fresh clone, with one documented command.
- Your analysis detects the problems you seeded in the data — closing the loop between Step 3 and Step 5.
- Your hypothesis is falsifiable, and you say plainly whether the data confirmed or refuted it.
- The one-screen visual communicates the finding without you in the room to explain it.
- Your leadership note reads like it was written by someone who has sat across from a senior official — and is honest about the limits of synthetic data.

5 · Constraints & guardrails

- **Synthetic data only.** Using or attempting to source real NHA / AB PM-JAY / ABDM data is an automatic disqualification.
- **No paid infrastructure.** Everything is doable locally and free. Don't spend money — it earns no credit and won't be reimbursed.
- **Time-box yourself.** 24 hours is the submission window, not the effort target. We'd rather see 4 sharp hours than 20 frantic ones.
- **Keep it private.** Don't post your repo publicly — share it with the panel instead (Section 8), so later candidates can't find your submission. Put nothing real or sensitive in it regardless; synthetic only.

6 · A note on privacy (it matters for this role)

PM-JAY claims are individual-level records, and the HAAU handles scheme data under the DPDP Act 2023 and the Strategy for Artificial Intelligence in Healthcare for India (SAHI). You are not asked to build an anonymisation pipeline in this short exercise — but your leadership note (D6) should show you are thinking like someone who will: keep your outputs at the district level, avoid any result that could single out an individual, and flag in one line where your synthetic dataset would raise a privacy concern if it were real.

7 · On using AI (please read — this is not a trap)

Modern AI tooling is central to how this kind of analytics-and-AI work gets done, so use it. Claude, Copilot, Cursor, ChatGPT — whatever you reach for. Vibe-coding is fine. We are not testing whether you can write pandas or SQL from memory.

What we *are* testing is whether you can direct the tools like someone with experience: catching the plausible-but-wrong query, noticing when a generated chart says something the data doesn't support, knowing that an LLM will happily compute a ratio off a column full of seeded nulls. Experience shows in the corrections, not the first draft.

So include a short `AI_LOG.md` **(D7)**: which tools you used, and — more usefully — two or three moments where the AI got it wrong or missed the NHA context, and how you caught and fixed it. A candidate who used AI heavily and shows sharp oversight scores higher than one who claims they wrote everything by hand.

8 · Submission

Organise your work as a single repository containing the deliverables below (use the deliverable IDs as file names or section headings so everything is traceable end to end). **Keep the repository private** — if you share a GitHub link, add the panel as collaborators rather than making it public, so that later candidates can't find your submission. A repository name of your own choosing is fine; there is no required naming convention.

ID	File / Path	Contents
D1	`/writeup/` — Problem_Breakdown	Step 2 — your decomposition of the question: sub-questions, metrics, confounders, data-quality traps.
D2	`/data_generator/` + `SEEDED_ISSUES.md`	Synthetic-data generator (`.py`) and a note documenting the flaws you deliberately seeded.
D3	`/writeup/` — Hypothesis	Step 4 — your falsifiable hypothesis, with confirm / refute conditions.
D4	`/analysis/` (`.ipynb` / `.py`)	Step 5 — analysis code: load, clean, build metrics, test the hypothesis.
D5	`/analysis/` (`.png` / `.html` / `.pdf`)	Step 6 — the one-screen visual / mini-dashboard.
D6	`/writeup/` — Leadership_Note	Step 6 — ⩽ 250-word note to leadership, incl. hypothesis, finding, decision, and what you would not claim.
D7	`AI_LOG.md`	Section 7 — tools used, and 2–3 places you caught the AI.
—	`SUBMISSION.md`	The cover sheet — see below.
—	`README.md`	How to run the whole thing end-to-end from a fresh clone.

You may keep the three short write-ups (D1, D3, D6) as one polished document (a single `.docx` or `.md`) if you prefer — the reasoning and document craft are assessed. The operational files (`AI_LOG`, `SUBMISSION`, `README`) stay as plain `.md`.

`SUBMISSION.md` must briefly include:

- One-command (or clearly-numbered) instructions to reproduce the thread from a fresh clone.
- A self-scored checklist against Section 4 (be honest — over-claiming is visible and costs you).
- Any assumptions you made — especially where the brief was ambiguous and you proceeded anyway.
- What you cut and why — the single most informative section for us.
- Approx. hours spent.

How to submit — reply to the email through which you received this exercise (your recruitment point of contact), attaching a **`.zip` export** of your repository (`git archive -o submission.zip HEAD`, or simply zip the folder). The ZIP is the frozen copy we review. If you

would also like us to see your commit history, include a link to a **private** repository with the panel added as collaborators. Please do not send your submission to the clarifying-questions address in Section 9 — that inbox is for questions only.

9 · How you'll be assessed

Your submission is scored against a structured rubric across five dimensions: **Hands-on Execution**, **Analytical Judgement & Hypothesis Quality**, **Data Handling & Cleaning**, **Problem-Solving & AI Direction**, and **NHA Context & Communication**. Strong performance advances you to a live technical round where we'll walk through your repo together and probe the choices you made.

Clarifying questions during the window are welcome — ambiguity is deliberate in places, and how you resolve it is part of what we watch. Send questions to **pchoudhary@wjcf.in**. This address is for **questions only** — your actual submission goes to your recruitment point of contact, as described in Section 8. If you don't hear back in time, don't stall: make a reasonable assumption, proceed, and state it explicitly in your submission. Working sensibly around missing information is itself a scored signal. Good luck.